

DIGITAL ELECTRONIC CIRCUITS : Term Project Report

UART PROTOCOL

Project Title: Design and Implementation of UART Protocol on FPGA using Verilog HDL

Team Details: Devabarkina Pratibha 24EC01016

Sangiseti Sita Mahalakshmi 24EC01046

Project Submission Date: 14/05/2026

Report Submission Date: 14/05/2026

Abstract: This project implements a UART communication system on FPGA using Verilog HDL. It includes transmitter and receiver and other required modules for smooth data processing. The system can receive data, process it by converting uppercase letters to lowercase and performing digit addition, and retransmit the modified data.

Objectives:

- To design and implement UART communication protocol using Verilog HDL.
- To simulate and test the design using Xilinx Vivado and hardware implementation.
- To gain practical understanding of FPGA-based digital system design.

System Requirements:

- **Hardware:** FPGA board (Basys3) , USB cable, PC/Laptop
- **Software:** Xilinx Vivado, PuTTY (for serial monitoring)

UART Protocol theory and implementation:

UART means **Universal Asynchronous Receiver and Transmitter** Protocol which is used for **serial communication** and can be applied to any transmitter and receiver. The word Asynchronous means that we **cannot use clock signal** for communication of data.

Basically, two devices are connected using wires and one is Transmitter (which sends the data) and the other is Receiver (which receives data). Here we're using FPGA and PC for communicating data.

UART Data format:



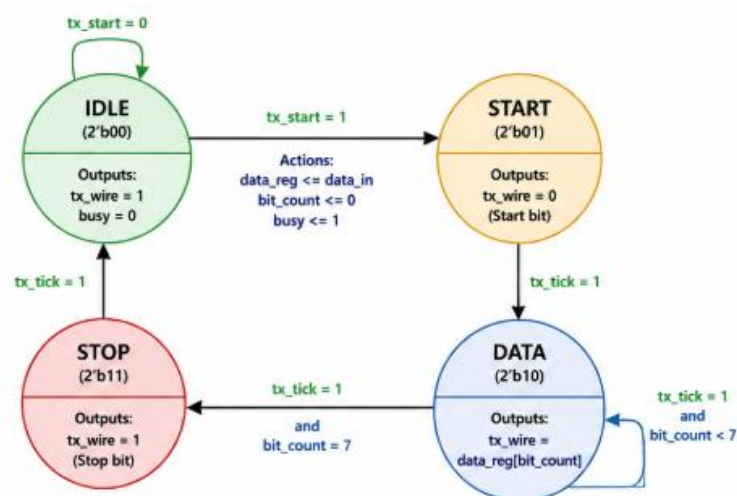
Reading left to right; this is the frame format of UART. At the beginning, the line normally stays at logic 1 i.e. nothing is being transmitted hence idle state. When the signal drops LOW it tells that data is incoming, it is the start state. After the start bit, the state enters data state where actual bits are sent. Here we're sending 8 bits, each bit is sent serially. The parity bit is used for error-checking, helps detect if one bit changed during transmission, anyhow we're not using this parity bit. When the line returns 1 i.e. stop bit is HIGH (stop state) , the transmission is complete and UART returns to idle state again.

The Baud Rate Generator synchronizes UART communication by generating accurate timing signals for transmitting and receiving data at the same selected baud rate. The FPGA clock is of 100MHz and the baud rate that is taken is 9600bps. Transmitter (TX) uses this pulse to know when to send the next bit and Receiver (RX) uses this pulse to know when to read/ sample the incoming bit.

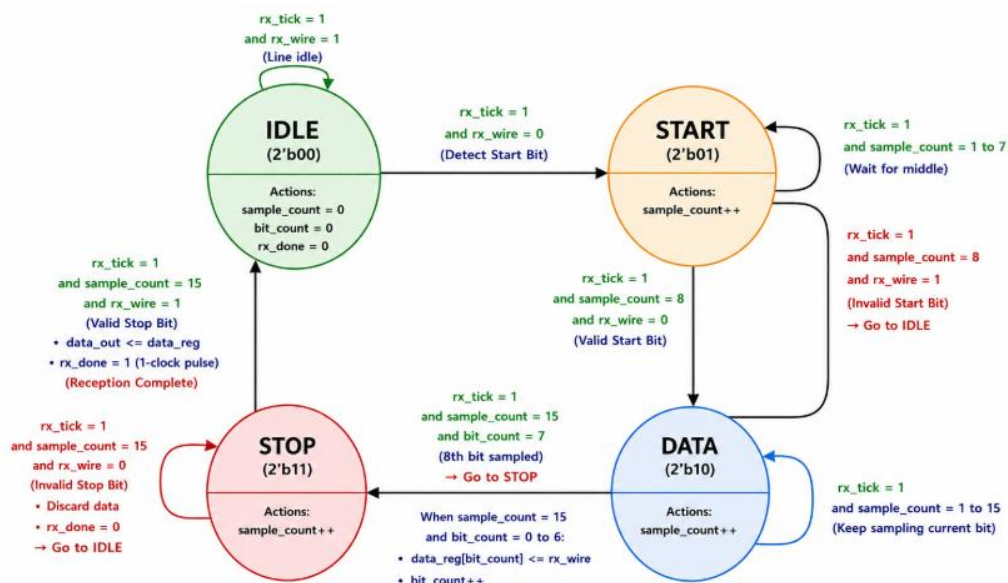
Module explanation: Inputs are FPGA clock and active-low reset, Outputs are tx_tick and rx_tick as the timing pulse for transmitter and receiver. Uses tx_counter which counts system clock cycles for the transmitter, when reaches 10415, resets to 0 and generates tx_tick pulse (bit is transmitted). Similarly, rx_counter counts 650, it resets to zero and generates rx_tick pulse(oversampling) (receive bit).

Transmitter (TX) Module: This transmits 8-bit serial data. It first converts parallel input data (data_in) into serial output (tx_wire) using FSM which is controlled by the baud rate timing signal (tx_tick). The

busy signal indicates that the transmitter is still sending data. The state diagrams of TX can explain everything that has written in the code.



Receiver (RX) Module: It is responsible for receiving serial data through the UART protocol and converting it into 8-bit parallel data, using FSM and receiver timing signal (rx_tick) to sample and



reconstruct the transmitted data accurately. The rx_done indicates that the data reception is successfully done. Similarly the state diagram for RX module can explain everything that has written in the code.

Debounce Module: This is used to remove unwanted noise from a mechanical push button signal. When a push button is pressed, the signal may rapidly toggle between 0 and 1 for a short time, which causes multiple fast triggers. This module ensures that only one clean pulse is generated for each button press. This module filters and generates a single stable output pulse. The input btn is raw push button given as input where as btn_pulse is debounced single-clock pulse output.

The button input (btn) is first stored in bit_sync to synchronize it with the system clock. This signal is compared with the previous state button state (btn_state). We're using counter to check the stability, if the signal changes, counter increments its value. The new button state is accepted only if it remains stable for 999999 clock cycles. When the valid button press (btn_sync=1) is confirmed, then the only the module generates a single pulse (btn_pulse=1) for one clock cycle.

UART Top Module: Here comes the complete integration of the system. It controls the complete data transmission and reception process. It combines baud generation, transmission, reception, button debouncing, data processing and FIFO buffering into one complete top module.

FIFO Buffer Usage: An 8-byte FIFO (First In First Out) memory is used to temporarily store processed received data. Sometimes new data may be received while transmitter is busy and immediate transmission of such data may not be possible hence; FIFO solves this problem by storing data in order until TX becomes free to transmit the data. In these pointers are used for indices: write_ptr which indicates where new data is to be stored, read_ptr indicates which stored data is to be transmitted next, whereas fifo_count keeps track of number of stored bytes.

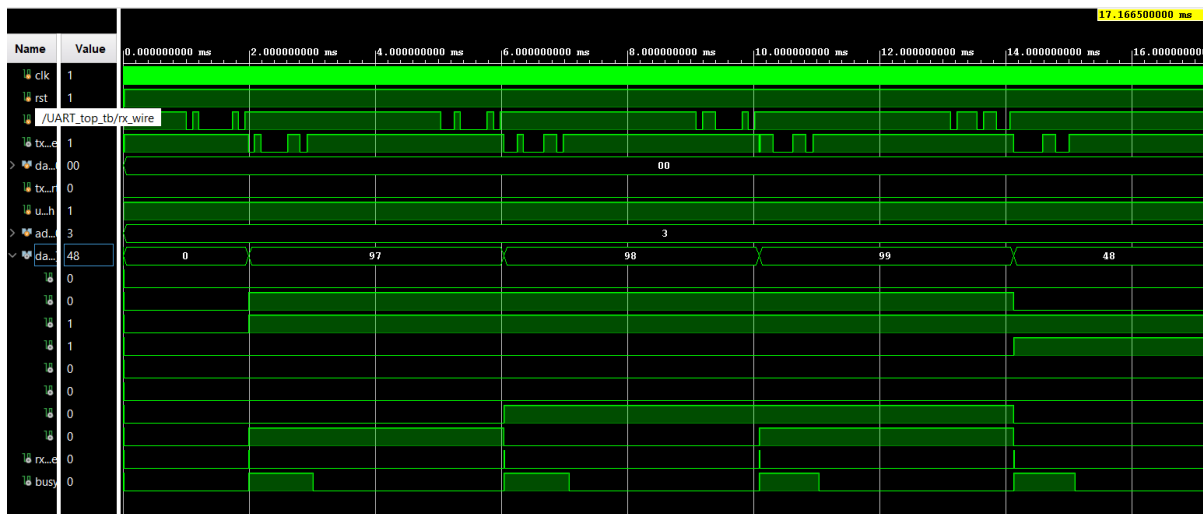
The basic operation in FIFO is: when rx_done=1, the processed received data is written into FIFO and when the transmitter is free (busy=0), data is read from FIFO and transmitted. So that this prevents data loss to an extent. The data processing logic : We have processed the data using two operations.

One is Uppercase to Lowercase Conversion: This is controlled by an input uart_case_switch, if the switch is ON and the ASCII value of the character is between 65 and 90 (A-Z) by adding 32 to the rx_data.

The other is Numeric Addition Operation: If received data is a digit i.e. ASCII values in between 48 and 57 of digits (0-9). It converts the ASCII value to actual number and adds value and uses modulo 10 to keep the result between 0-9 and convert back to ASCII.

Transmission logic: There are two ways it can start, one is manually using push button (tx_start_pulse) and other is automatic when FIFO contains processed received data. Using the if condition: if (! busy && fifo_count >0) i.e. fifo is not fully empty and data can be transmitted.

Simulation Results: The output waveform for the written test bench code:



Hardware Implementation:

- i. The complete Verilog design is synthesized, implemented and programmed onto FPGA using Xilinx Vivado. The generated bit stream file is loaded into the FPGA through USB connection.
- ii. The input and output planning is done using xdc constraint file.
- iii. The FPGA communicates with the PC through the onboard USB-UART interface, we have used a serial terminal software named PuTTY to monitor the data transmitted and received.
- iv. The UART settings for this is 8N1: Baud rate: 9600bps, 8 Data bits, No Parity bit, 1 Stop bit and flow control as none. The COM port corresponds to the FPGA board selected in the terminal software can be found from the device manager of the settings inside PC.
- v. The implemented system is tested by sending and receiving characters through the serial terminal.
- vi. The transmitted and processed output is displayed on the terminal.

Results:

- i. Both the transmission and reception data appeared correctly.
- ii. The Uppercase to Lowercase conversion did work properly.
- iii. The Digit Addition operation also worked.
- iv. FIFO and debounce worked and helped reducing the errors.
- v. Simulating waveforms was successful.
- vi. FPGA implementation was also successful.

Conclusion: The Project provided practical experience in digital communication using hardware system. It helped in understanding the complete design flow, from coding each module to integrating them into a top-level system and hardware testing. This project also highlighted the importance of timing accuracy, synchronization, and efficient data handling. Using FIFO helped us prevent data loss, handle speed mismatch and maintained data order. Our faculty also explained the clear usage of FIFO that when the transmitter and receivers are of different clock frequency we can't directly transmit or receive data, there'll be time synchronization issues; but instead, we'll be using this FIFO method to control the data flow in each.

Future Scope:

- i. Implementation of advanced UART features.
- ii. Implementation of SPI Protocol
- iii. Implementation of I2C Protocol
- iv. Other embedded and communication protocols

References:

- i. [Universal Asynchronous Receiver Transmitter \(UART\) Protocol - GeeksforGeeks](#)
- ii. [UART Protocol Introduction | Basics of Serial Communication Explained || All about VLSI ||](#)
- iii. [embedded - Receive/Transmit FIFO vs Data Registers in UART - Stack Overflow](#)
- iv. [A novel approach to ensure efficient asynchronous communication using FIFO-based UART module | International Journal of Information Technology | Springer Nature Link](#)

GitHub Link for codes and readme file:

<https://github.com/dpratibha0753-wq/UART-Protocol-Basys3>